Project Title

Regex to NFA Converter using Thompson's Construction

Team Members

- Omar Shoulkamy – 231000802
- Kamel Walid Ramdan – 231002427
- Mohamed gad – 231000546

Project Description

This project implements a converter that transforms a **Regular Expression (Regex)** into an equivalent **Non-deterministic Finite Automaton (NFA)** using **Thompson's Construction Algorithm**, a fundamental technique in the Theory of Computation.

The system parses a given regular expression, converts it into postfix notation, and incrementally builds the corresponding NFA by combining smaller automata structures. The final output includes a complete representation of the NFA, including its states, transitions, diagram, and formal definition. This allows users to clearly visualize how theoretical regex expressions are translated into machine-executable automata.

What Does It Do?

- Accepts a **regular expression** as input.
- Converts it to **postfix notation**.
- Constructs an **NFA** using Thompson's rules.
- Displays:
    - Start and end states
    - Transition list
    - ASCII diagram of the NFA

- Formal definition ($M = (Q, \Sigma, \delta, q_0, F)$)
- Transition function in **table form**

Input Format

- A valid **regular expression** using:
  - | → Union
  - * → Kleene Star (zero or more)
  - + → One or more
  - ( ) → Grouping
  - Concatenation is implicit

Example:

```
(a|b)*
```

Output Format

The program outputs:

1. Postfix Expression

```
ab|*
```

2. NFA Components

- Start state
- Accept (final) state
- Transition list

3. NFA Diagram

- ASCII visualization of states and transitions

4. Formal Definition

```
M = (Q, Σ, δ, q0, F)
```

Example:

```
Q = {'S0', 'S1', 'S2', ...}
Σ = {'a', 'b'}

δ:
(S0, a) → {S1}
(S1, ε) → {S2}
...

q0 = S6
F = {'S7'}
```

5. Transition Table

State	a	b	ε
S0	{S1}	∅	∅
S1	∅	∅	{S5}
...			

Inside Mechanism

1. Regex to Postfix Conversion

- The input regex is converted into postfix form using operator precedence.
- This simplifies parsing and allows stack-based processing.

2. Thompson's Construction Algorithm

The NFA is built using the following rules:

- **Symbol (a)** → simple 2-state NFA

- **Concatenation (AB)** → connect end of A to start of B
- **Union (A | B)** → split using ε-transitions
- **Kleene Star (A*)** → loop with ε-transitions
- **Plus (A+)** → one or more repetitions

3. State Representation

- Each state is uniquely identified (S0, S1, ...).
- Transitions are stored as:

```
symbol → list of next states
```

4. Output Generation

The system generates:

- Raw transitions
- ASCII diagram
- Formal mathematical definition
- Structured transition table

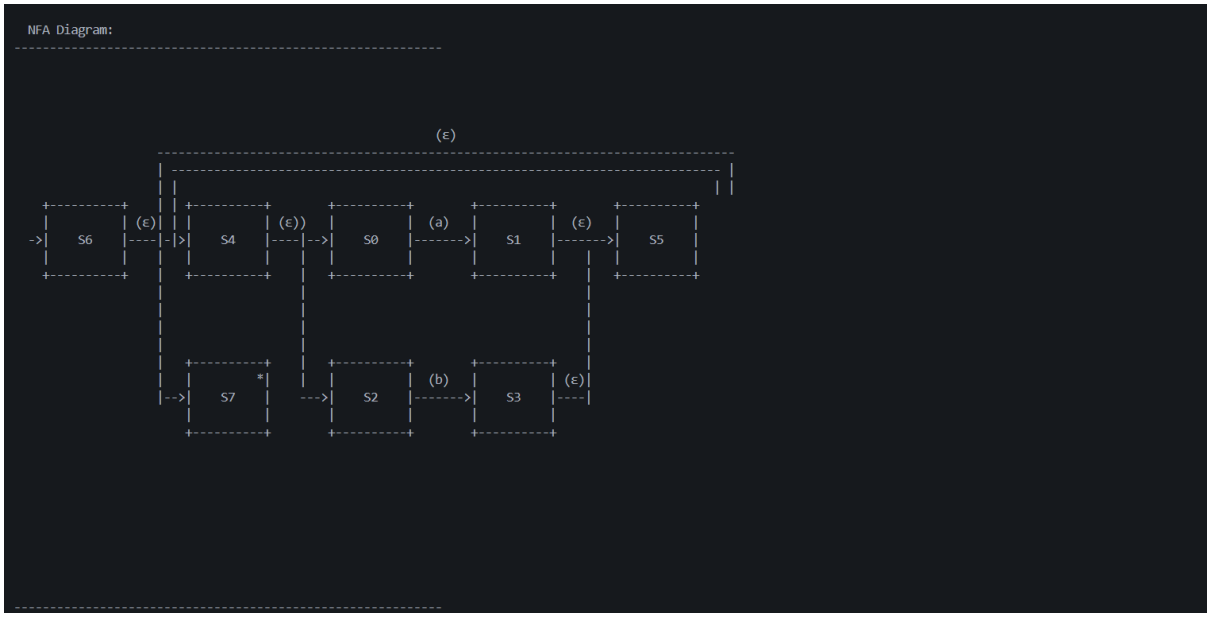
Programming Language, Tools & Libraries

- **Language:** Python
- **Tools:** Command Line Interface (CLI)
- **Libraries:**
    - Only built-in Python features (no external libraries)

Project Output (Example Run)

```
PS C:\Users\omare\OneDrive\Desktop\Theory-Projects> python -u "c:\Users\omare\OneDrive\Desktop\Theory-Projects\1\main.py"
 ==================================================
       Regex to NFA Converter
       Using Thompson's Construction
 ==================================================
   Supported operators:
     |   -> Union
     *   -> Kleene Star (zero or more)
     +   -> One or more
     ()  -> Grouping
     Concatenation is implicit
 ==================================================

   Enter regex (or 'q' to quit): (a|b)*

   Postfix form : ab|*

   Start State : S6
   End State   : S7
   Transitions :
     S0 --[a]--> S1
     S1 --[ε]--> S5
     S2 --[b]--> S3
     S3 --[ε]--> S5
     S4 --[ε]--> S0
     S4 --[ε]--> S2
     S5 --[ε]--> S4
     S5 --[ε]--> S7
     S6 --[ε]--> S4
     S6 --[ε]--> S7

   NFA Diagram:
 ----------------------------------------------------------
```

```
Regex: (a|b)*

Postfix: ab|*

Start State : S6
End State   : S7

Transitions:
S0 --[a]--> S1
S1 --[ε]--> S5
...
```



M = (Q, Σ, δ, q0, F)

Q = {'S3', 'S7', 'S6', 'S1', 'S0', 'S4', 'S5', 'S2'}
Σ = {'a', 'b'}

δ:
(S6, ε) → {S4}
(S6, ε) → {S7}
...

q0 = S6
F = {'S7'}

Transition Table:

State	a	b	ε
S0	{S1}	∅	∅
S1	∅	∅	{S5}
S2	∅	{S3}	∅

...

```
-----------------------------------------------------------
  NFA FD:

M = (Q, Σ, δ, q0, F)

Q = {'S3', 'S7', 'S6', 'S1', 'S0', 'S4', 'S5', 'S2'}
Σ = {'a', 'b'}

δ:
(S6, ε) → {S4}
(S6, ε) → {S7}
(S4, ε) → {S0}
(S4, ε) → {S2}
(S0, a) → {S1}
(S2, b) → {S3}
(S1, ε) → {S5}
(S3, ε) → {S5}
(S5, ε) → {S4}
(S5, ε) → {S7}

q0 = S6
F = {'S7'}

Transition Function δ (Table Form):

State   a       b       ε
S0      {S1}    ∅       ∅
S1      ∅       ∅       {S5}
S2      ∅       {S3}    ∅
S3      ∅       ∅       {S5}
S4      ∅       ∅       {S0,S2}
S5      ∅       ∅       {S4,S7}
S6      ∅       ∅       {S4,S7}
S7      ∅       ∅       ∅
-----------------------------------------------------------
```

Conclusion

This project successfully demonstrates how regular expressions can be formally transformed into NFAs using Thompson's construction. It bridges theoretical concepts with practical implementation and provides multiple representations (diagram, formal definition, and table) to enhance understanding and clarity.